DAM

PROMETEO
by thePower

Tema 1: **Programación Multiproceso**

Programación de Servicios y Procesos · 2º DAM

Olga M. Moreno Martín

10 / 09 / 2026

PROMETEO
by thePower



Toda la documentación de esta asignatura queda recogida bajo la licencia de Creative Commons
Olga M. Moreno Martín

https://creativecommons.org/licenses/by-nc-nd/4.0/

En el caso de incumplimiento o infracción de una licencia Creative Commons, el autor, como con cualquier otra obra y licencia, habrá de recurrir a los tribunales. Cuando se trate de una infracción directa (por un usuario de la licencia Creative Commons), el autor le podrá demandar tanto por infracción de la propiedad intelectual como por incumplimiento contractual (ya que la licencia crea un vínculo directo entre autor y usuario/licenciatario). El derecho moral de integridad recogido por la legislación española queda protegido aunque no aparezca en las licencias Creative Commons. Estas licencias no sustituyen ni reducen los derechos que la ley confiere al autor; por tanto, el autor podría demandar a un usuario que, con cualquier licencia Creative Commons, hubiera modificado o mutilado su obra causando un perjuicio a su reputación o sus intereses. Por descontado, la decisión de cuándo ha habido mutilación y de cuándo la mutilación perjudica la reputación o los intereses del autor quedaría en manos de cCutacia Juez o Tribunal.

PROMETEO
by thePower

SECCIÓN 1

Introducción a la Programación Concurrente y Paralela

01

TEMA · EXPLICACIÓN

PROMETEO by thePower

CONCEPTO

Paralelismo y concurrencia

Paralelismo: Varios objetos realizando una acción cada uno simultáneamente (síncrono).
Síncrono → Sincronización, todos a la vez



Concurrencia: Un solo objeto, con varias tareas “activas” entre las que va alternando (asíncrono).
Asíncrono: Es capaz de llevar el control de muchas tareas diferentes y es capaz de alternar entre ellas.

TEMA · EXPLICACIÓN

PROMETEO by thePower

CONCEPTO

Concurrencia

COMO UN CAMARERO/A

- Si vas a comer a un bar y te sientas en la mesa, si dicho bar tiene 12 mesas, puede que ese bar disponga de 3 camareros, los cuales se encargarán cada uno de 4 mesas.
- Cuando te sientes, el camarero correspondiente te dará la carta y solicitará las bebidas.
- Cuando te sirva las bebidas, te dará tiempo para pensar que vas a comer y, mientras eliges, se va a atender a otra mesa.
- Esto es concurrencia, es capaz de llevar varios clientes a la vez y servirlos a todos por igual.
- Para aprovecharlo, tiene que haber varios clientes.
- Un cliente no termina más rápido por asignarle un camarero exclusivo.
- Cada cliente termina a su ritmo.

TEMA · EXPLICACIÓN

PROMETEO by thePower

CONCEPTO

Tipos de programación

- La programación concurrente hace referencia a la capacidad de un sistema para ejecutar múltiples tareas de manera simultánea, aprovechando al máximo los recursos disponibles. Esto se logra mediante la interleaving de tareas, donde el sistema alterna entre diferentes tareas en ejecución, dando la impresión de que se están ejecutando al mismo tiempo.

- La programación paralela, por otro lado, implica la ejecución simultánea de múltiples tareas en diferentes núcleos de procesamiento, lo que puede mejorar significativamente el rendimiento en sistemas con múltiples procesadores.

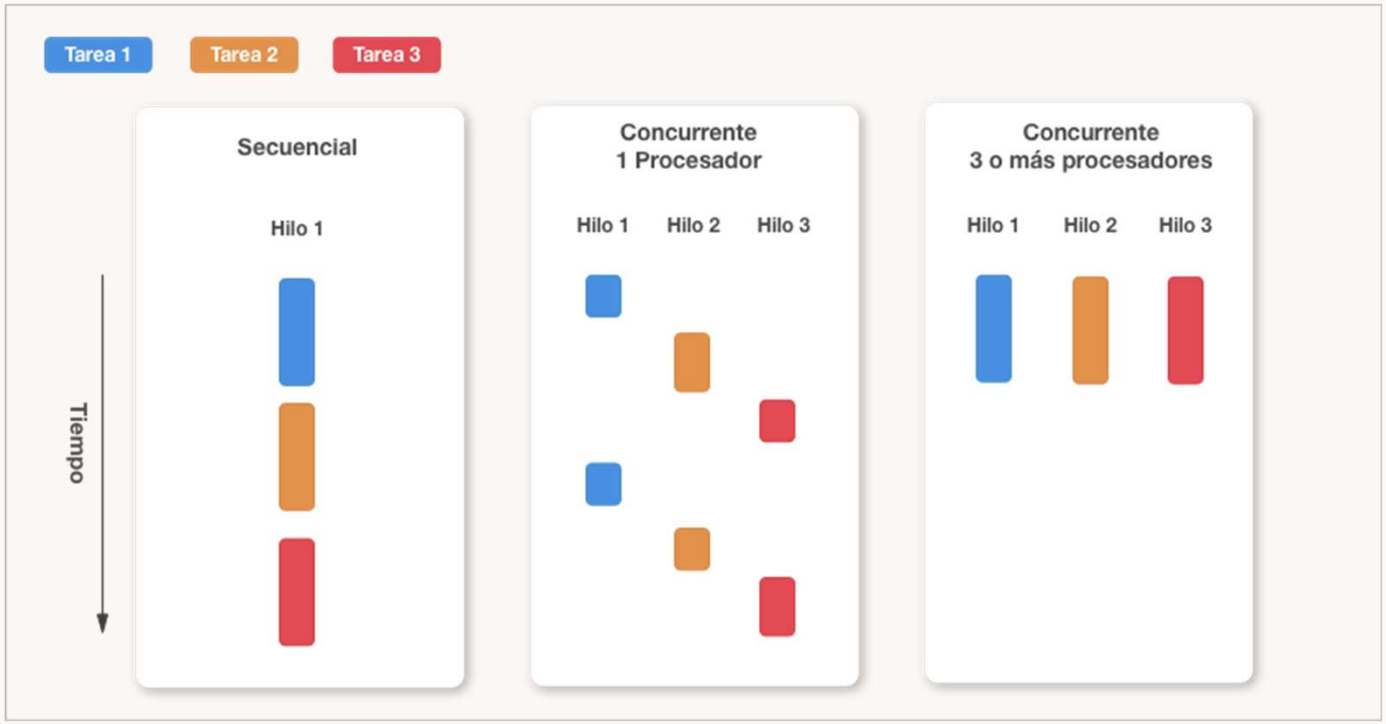
- Esto es posible gracias a que a día de hoy los procesadores cuentan con múltiples núcleos, lo que permite la ejecución de múltiples hilos de manera simultánea.

TEMA · EXPLICACIÓN

Diferencias

PROMETEO by thePower

CONCEPTO



TEMA · EXPLICACIÓN

PROMETEO by thePower

CONCEPTO

Núcleo vs hilo

- Es habitual a la hora de hablar de procesadores, referirse a los núcleos y hilos como si fueran lo mismo, pero en realidad son conceptos diferentes.
- Un **núcleo** es la unidad física de procesamiento en un CPU, mientras que un **hilo** es una unidad de ejecución dentro de un proceso. Un núcleo puede ejecutar múltiples hilos a través de técnicas como el hyper-threading.
- Es por esto que es habitual ver términos como "núcleo" e "hilo" utilizados indistintamente, aunque se refieren a conceptos diferentes.

PROMETEO by thePower

CONCEPTO

Características de la programación concurrente

La programación concurrente tiene una serie de características especiales o diferentes con respecto a la programación más tradicional:

- Interleaving: Las tareas se ejecutan de manera intercalada, lo que permite que el sistema responda a múltiples eventos al mismo tiempo.
- Sincronización: Es necesario coordinar el acceso a recursos compartidos para evitar condiciones de carrera y garantizar la consistencia de los datos.
- Desempeño: La programación concurrente puede mejorar el rendimiento al aprovechar mejor los recursos del sistema, especialmente en sistemas con múltiples núcleos.
- Escalabilidad: Las aplicaciones concurrentes pueden escalar mejor en entornos distribuidos, donde múltiples instancias de una aplicación pueden ejecutarse en diferentes nodos.

TEMA · EXPLICACIÓN

PROMETEO by thePower

CONCEPTO

Definiciones importantes

¿Qué son las condiciones de carrera?

- La condición de carrera o race condition se produce cuando dos o más hilos acceden a un recurso compartido y tratan de cambiarlo al mismo tiempo. Si el acceso no está debidamente sincronizado, el resultado final puede depender del orden en que se ejecutan los hilos, lo que lleva a comportamientos inesperados y errores difíciles de reproducir.

Entornos distribuidos

- Cuándo hablamos de entornos distribuidos en el ámbito de la programación nos referimos a sistemas en los que los componentes de una aplicación se ejecutan en diferentes máquinas, que pueden estar ubicadas en la misma red local o en diferentes partes del mundo. Estos sistemas distribuidos pueden comunicarse entre sí a través de redes, lo que les permite trabajar juntos para lograr un objetivo común.

- Un nodo de un sistema distribuido se refiere a una instancia individual de un componente de la aplicación que se ejecuta en una máquina específica. Cada nodo puede tener su propia memoria y recursos, y puede comunicarse con otros nodos a través de la red.

TEMA · EXPLICACIÓN

PROMETEO by thePower

CONCEPTO

Ámbitos de programación concurrente

La programación concurrente tiene múltiples ámbitos de aplicación, que incluyen:

- Sistemas operativos: La gestión de procesos y hilos en un sistema operativo es un ejemplo clásico de programación concurrente. Los sistemas operativos utilizan técnicas de planificación para permitir que múltiples procesos se ejecuten de manera concurrente, compartiendo recursos como CPU y la memoria.
- Aplicaciones web: En el desarrollo de aplicaciones web, la programación concurrente se utiliza para manejar múltiples solicitudes de usuarios simultáneamente. Los servidores web pueden utilizar hilos o procesos para atender cada solicitud de manera independiente, mejorando la capacidad de respuesta y el rendimiento.
- Sistemas distribuidos: En entornos distribuidos, la programación concurrente es esencial para coordinar la comunicación y el procesamiento entre diferentes nodos. Los sistemas distribuidos a menudo utilizan modelos de programación concurrente para gestionar la concurrencia y la sincronización entre componentes que se ejecutan en diferentes máquinas.
- Procesamiento de datos: En aplicaciones de procesamiento de datos, como el análisis de grandes volúmenes de información, la programación concurrente se utiliza para dividir tareas en subtareas más pequeñas que pueden ejecutarse en paralelo, aprovechando al máximo los recursos del sistema.
- Interacción con el usuario: En aplicaciones con interfaces gráficas, la programación concurrente se utiliza para mantener la interfaz de usuario receptiva mientras se realizan operaciones en segundo plano, como la carga de datos o la ejecución de cálculos intensivos.

A medida que los sistemas se vuelven más complejos y distribuidos, la necesidad de enfoques de programación concurrente se vuelve cada vez más importante.

TEMA · EXPLICACIÓN

PROMETEO by thePower

CONCEPTO

Programación paralela y distribuida

La programación paralela y distribuida son dos enfoques relacionados con la ejecución concurrente de tareas, pero tienen diferencias clave.

- La programación paralela se centra en la ejecución simultánea de múltiples tareas en un único sistema, aprovechando múltiples núcleos de CPU o procesadores para mejorar el rendimiento.
- En este enfoque, las tareas se dividen en subtareas que se pueden ejecutar en paralelo, lo que permite un procesamiento más rápido y eficiente. La programación paralela es especialmente útil en aplicaciones que requieren un alto rendimiento, como el procesamiento de imágenes, simulaciones científicas y análisis de datos.

- Por otro lado, la programación distribuida se refiere a la ejecución de tareas en múltiples máquinas o nodos que están conectados a través de una red.
- En este enfoque, los componentes de una aplicación se distribuyen en diferentes máquinas, lo que permite una mayor escalabilidad y flexibilidad. La programación distribuida es común en sistemas en la nube, aplicaciones web y entornos de microservicios, donde los recursos se pueden asignar y escalar dinámicamente según la demanda.

Ambos enfoques son fundamentales para el desarrollo de aplicaciones modernas y se utilizan en conjunto para abordar problemas complejos que requieren tanto paralelismo como distribución.

TEMA · EXPLICACIÓN

PROMETEO by thePower

CONCEPTO

Ventajas y desventajas de la programación paralela

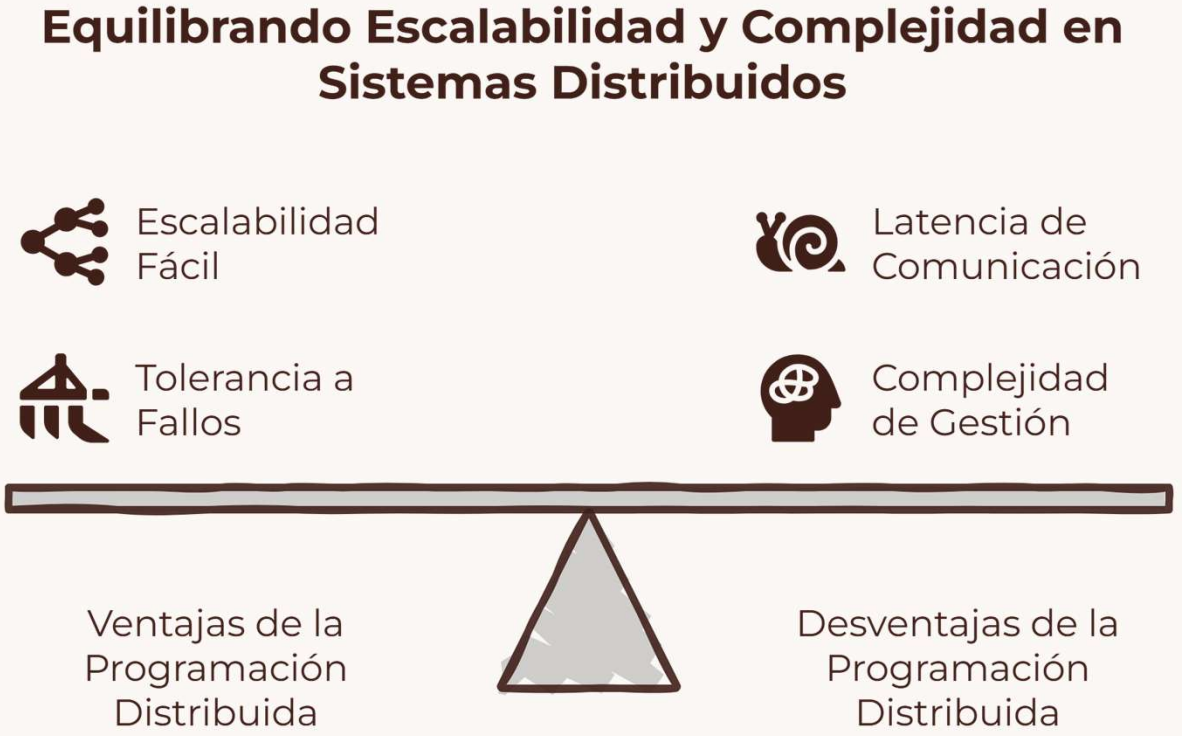


TEMA · EXPLICACIÓN

PROMETEO by thePower

CONCEPTO

Ventajas y desventajas de la programación distribuida



PROMETEO
by thePower

SECCIÓN 2

Procesos y subprocesos en los SSOO

02

TEMA · EXPLICACIÓN

PROMETEO by thePower

CONCEPTO

Procesos

- Un proceso es un programa en ejecución, que incluye el código del programa, sus datos y su estado de ejecución.

- Los sistemas operativos modernos permiten la ejecución de múltiples procesos de manera concurrente, lo que significa que varios procesos pueden estar en ejecución al mismo tiempo.



TEMA 1 · ACTIVIDAD

PROMETEO by thePower

Ejercicio 1: Ejecutar un comando simple

05 minutos

Enunciado: Escribe un programa en Kotlin que ejecute el comando ls y muestre la salida por pantalla.

Comandos:

- *bash ls*
- *PowerShell Get-ChildItem*
- *CMD dir*

Requisitos:

- *Usar ProcessBuilder.*
- *Mostrar la salida estándar del proceso.*

TEMA 1 · ACTIVIDAD

Ejercicio 2: Ejecutar un proceso con argumentos

PROMETEO by thePower

05 minutos

Enunciado: Modifica el ejercicio anterior para ejecutar el comando ls -l (listado detallado) y mostrar la salida.

Comandos: bash ls -l
PowerShell Get-ChildItem | Format-List ·
CMD dir /q (incluye el propietario de cada archivo)

Requisitos:

- *Pasar argumentos al proceso.*

TEMA 1 · ACTIVIDAD

PROMETEO by thePower

Ejercicio 3: Capturar Errores del proceso

05 minutos

Enunciado: Escribe un programa que ejecute un comando inexistente y capture tanto la salida estándar como la de error.

Comandos: lanza el comando inexistente a través del intérprete:
bash -c comando_inexistente

Requisitos:

- *Usar redirectErrorStream(false) y mostrar ambas salidas.*

TEMA 1 · ACTIVIDAD

Ejercicio 4: Esperar a que termine un proceso

PROMETEO by thePower

05 minutos

Enunciado: Escribe un programa que ejecute el comando sleep 2 y muestre un mensaje antes y después de la espera.

Comandos:
bash sleep 2

Requisitos:
Usar waitFor() para esperar la finalización.

TEMA · EXPLICACIÓN

Procesos

PROMETEO by thePower

CONCEPTO

- En el contexto de un sistema operativo podemos hablar de la gestión de procesos, que incluye la creación, planificación y terminación de procesos.
- Un proceso a nivel de sistema operativo se refiere a una instancia de un programa en ejecución, que tiene su propio espacio de direcciones y recursos asignados.
- Un mismo programa puede tener múltiples instancias en ejecución, cada una de las cuales se considera un proceso separado. Por ejemplo, un navegador web puede tener múltiples ventanas o pestañas abiertas, cada una de las cuales se ejecuta como un proceso independiente.
- En Windows podemos utilizar el Administrador de tareas para ver y gestionar los procesos en ejecución. Este herramienta nos permite ver información detallada sobre cada proceso, como su uso de CPU y memoria, y nos permite finalizar procesos que no responden.
- En sistemas Linux también podemos utilizar herramientas como top o htop para monitorizar y gestionar los procesos en ejecución.

TEMA · EXPLICACIÓN

PROMETEO by thePower

CONCEPTO

Subprocesos

- Los subprocesos, o hilos de ejecución, son unidades más pequeñas dentro de un proceso. Un proceso puede contener múltiples subprocesos que comparten el mismo espacio de direcciones y recursos, pero que se ejecutan de manera independiente. Esto permite una mayor eficiencia y una mejor utilización de los recursos del sistema.

- Los subprocesos son especialmente útiles en aplicaciones que requieren realizar múltiples tareas simultáneamente, como servidores web que manejan múltiples conexiones de clientes al mismo tiempo. Al utilizar subprocesos, un servidor puede atender a varios clientes sin bloquearse en una sola tarea.

- Los subprocesos también son utilizados en aplicaciones de procesamiento de datos, donde se pueden dividir tareas grandes en subtareas más pequeñas que se pueden ejecutar en paralelo. Esto puede mejorar significativamente el rendimiento y la capacidad de respuesta de la aplicación.

TEMA · EXPLICACIÓN

PROMETEO by thePower

CONCEPTO

Estados de un proceso

Un proceso puede encontrarse en diferentes estados a lo largo de su ciclo de vida. Los estados más comunes son:

- Nuevo: El proceso está siendo creado.
- Listo: El proceso está preparado para ejecutarse y espera ser asignado a un procesador.
- Ejecutando: El proceso está en ejecución y utilizando el CPU.
- Bloqueado: El proceso está esperando un evento o recurso, como la finalización de una operación de entrada/salida.
- Terminado: El proceso ha finalizado su ejecución y ha liberado sus recursos.

Estos estados pueden variar ligeramente entre diferentes sistemas operativos, pero en general, representan el ciclo de vida de un proceso en un sistema operativo.



TEMA · EXPLICACIÓN

PROMETEO by thePower

CONCEPTO

Gestión y Monitorización de Procesos

Existen diversos programas y comandos para la gestión y monitorización de procesos en diferentes sistemas operativos. Algunos de los más comunes son:

- Windows: El Administrador de tareas permite ver y gestionar los procesos en ejecución. También se pueden utilizar comandos como tasklist y taskkill en la línea de comandos.
- **Linux:** Herramientas como top, htop y ps permiten monitorizar los procesos en tiempo real. Para gestionar procesos, se pueden utilizar comandos como kill y nice.
- macOS: El Monitor de Actividad proporciona una interfaz gráfica para gestionar procesos. También se pueden utilizar comandos similares a los de Linux en la terminal.

Estas herramientas son esenciales para los administradores de sistemas y los desarrolladores, ya que permiten identificar y resolver problemas relacionados con el rendimiento de los procesos.

TEMA · ACTIVIDAD

PROMETEO by thePower

Ejercicio 5: Listar procesos activos

05 minutos

Enunciado: Escribe un programa que ejecute el comando ps y muestre la lista de procesos activos.

Comandos: bash ps

Requisitos:

Leer y mostrar la salida del proceso.

TEMA · ACTIVIDAD

PROMETEO by thePower

Ejercicio 6: Lanzar varios procesos en paralelo

Enunciado: Escribe un programa que lance tres procesos que ejecuten sleep 1, sleep 2 y sleep 3 en paralelo y muestre cuándo termina cada uno.

Comandos: bash sleep N

Nota: El uso de hilos (thread) se verá en detalle en el siguiente tema. Aquí se muestra como ejemplo avanzado para quien quiera explorar cómo lanzar procesos en paralelo desde Kotlin.

Requisitos:

Usar hilos para lanzar los procesos en paralelo.

TEMA · EXPLICACIÓN

PROMETEO by thePower

CONCEPTO

Algoritmos de planificación

Cuando varios procesos compiten por el mismo recurso, el sistema operativo debe decidir cuál de ellos obtiene acceso al recurso y por cuánto tiempo. Este proceso de toma de decisiones se conoce como planificación de procesos.

Cada uno de estos algoritmos tiene sus propias características y es adecuado para diferentes tipos de sistemas y cargas de trabajo. La elección del algoritmo de planificación adecuado es crucial para garantizar un rendimiento óptimo del sistema.

Un mismo proceso puede tener varias ráfagas de CPU, es decir, puede ser interrumpido y reanudado varias veces durante su ejecución. Las razones por las cuales un proceso puede necesitar un cierto tiempo de bloqueo pueden ser como que necesite realizar operaciones de entrada/salida o esperar la finalización de otro proceso.

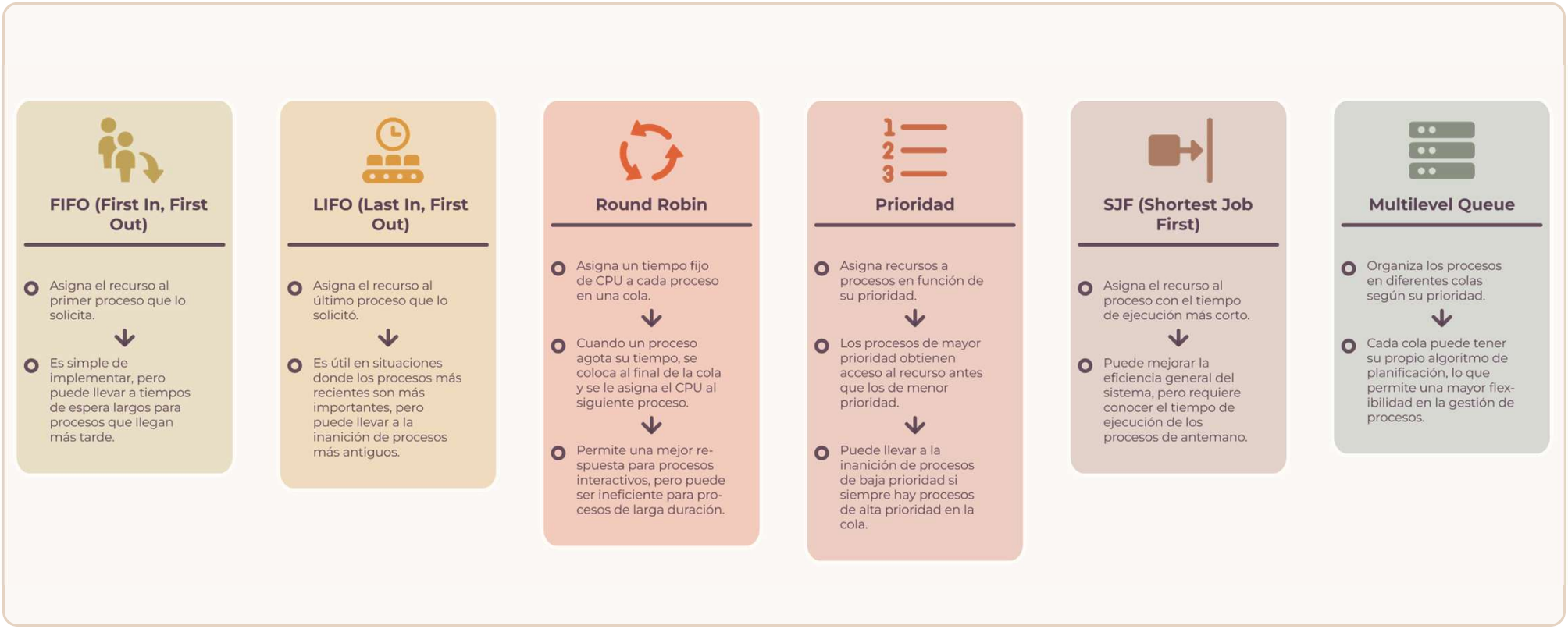
Los algoritmos de planificación pueden ser expropiativos o no expropiativos. Los algoritmos expropiativos permiten que un proceso en ejecución sea interrumpido para dar paso a otro proceso de mayor prioridad. Por otro lado, los algoritmos no expropiativos no permiten que un proceso en ejecución sea interrumpido; debe completar su tiempo de CPU antes de que otro proceso pueda ejecutarse.

TEMA · EXPLICACIÓN

PROMETEO by thePower

CONCEPTO

Algoritmos de planificación



TEMA · EXPLICACIÓN

PROMETEO by thePower

CONCEPTO

El concepto de Quantum

- El quantum es el tiempo máximo que un proceso puede utilizar el CPU antes de ser interrumpido y colocado de nuevo en la cola de listos.

- Este concepto es fundamental en algoritmos de planificación como Round Robin, donde se asigna un quantum fijo a cada proceso. La elección de un quantum adecuado es importante: un quantum muy corto puede llevar a una alta sobrecarga de conmutación de contexto, mientras que un quantum muy largo puede hacer que los procesos interactivos se sientan lentos.

- Algunos algoritmos de planificación utilizan el concepto de quantum para gestionar el tiempo de CPU de manera más eficiente. Por ejemplo, en el algoritmo Round Robin, se asigna un quantum fijo a cada proceso, lo que garantiza que todos los procesos tengan la oportunidad de ejecutarse en un tiempo razonable. Esto es especialmente útil en sistemas interactivos, donde la capacidad de respuesta es crucial.

PROMETEO
by thePower

SECCIÓN 3

Programación con Procesos y Sincronización

03

Sección 3.1

Creación y Gestión de Procesos 03.1

TEMA · EXPLICACIÓN

La clase Process

PROMETEO by thePower

CONCEPTO

La clase Process

La clase Process representa un proceso en ejecución. Esta clase proporciona métodos para interactuar con el proceso, como obtener su salida, error y código de salida.

En este ejemplo, creamos un proceso que ejecuta el comando ls -l en el directorio raíz. Utilizamos ProcessBuilder para configurar y lanzar el proceso, y luego leemos su salida.

La gestión de procesos incluye la capacidad de esperar a que un proceso termine, obtener su código de salida y manejar errores.

Process.kt

```kotlin
import java.io.BufferedReader
import java.io.File
import java.io.InputStreamReader

fun main(){
    val processBuilder = ProcessBuilder("bash","-c","ls -l")
    //Establecer el directorio de trabajo
    processBuilder.directory(File("C:\\Users\\Usuario\\Desktop\\2DAM-AS-2627\\Kotlin_Practicas\\src\\Tema1"))
    // Redirigir errores al flujo de salida
    processBuilder.redirectErrorStream(true)
    try {
        // Iniciamos el proceso, start devuelve un objeto Process
        val process = processBuilder.start()

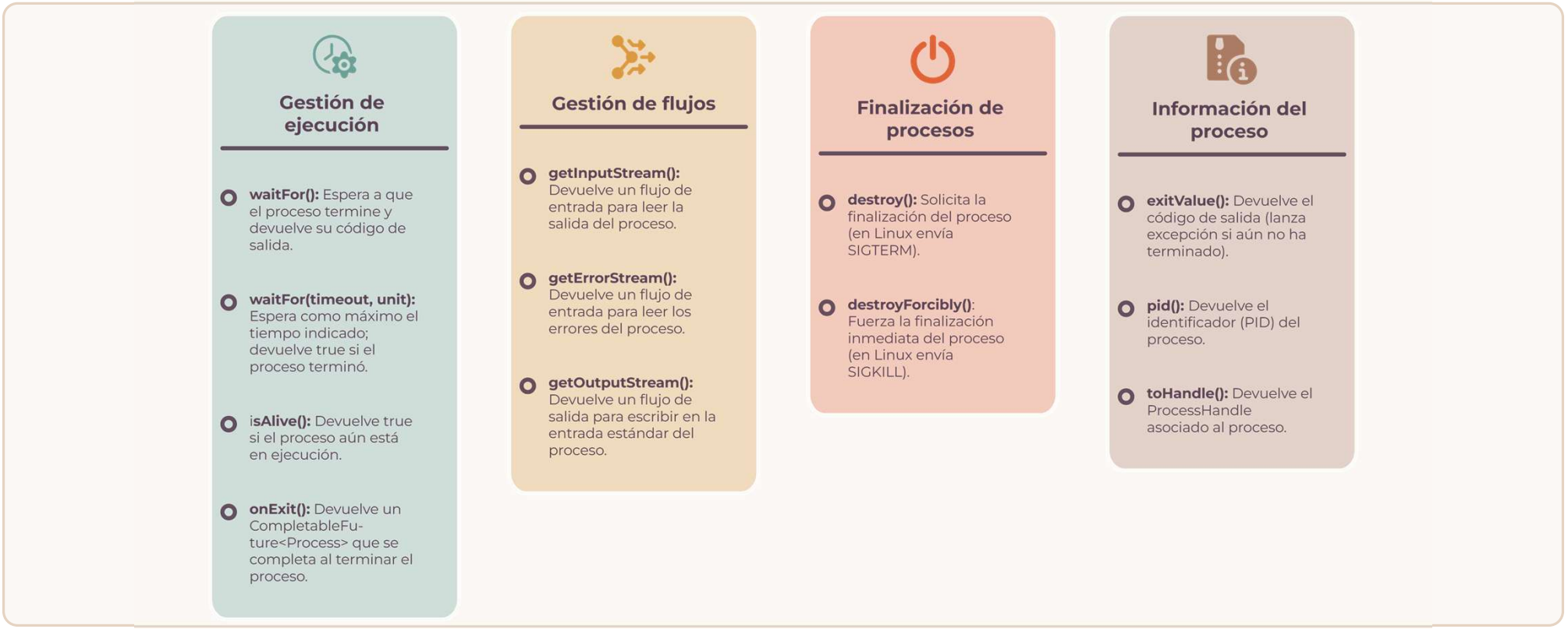
        //Leer la salida del proceso
        val reader = BufferedReader(InputStreamReader(process.inputStream))
        var line:String?
        while(reader.readLine().also {line = it}!= null){
            println(line)
        }
        //Esperar a que el proceso finalice
        val exitCode = process.waitFor()
        println("Proceso terminado: $exitCode")
    }catch(e: Exception){
        e.printStackTrace()
    }
}
```

TEMA · EXPLICACIÓN

PROMETEO by thePower

CONCEPTO

Métodos de Process



Ejemplos

Ir al proyecto Kiotlin_Practicas:

- Tema1.ProcessPs
- Tema1.ProcessGrep
- Tema1.ProcessSleep

Y observar el uso de los diferentes comandos y métodos de Process.

TEMA · EXPLICACIÓN

La clase ProcessBuilder

PROMETEO by thePower

CONCEPTO

La clase ProcessBuilder

- A diferencia de Process, la clase ProcessBuilder se utiliza para crear y configurar un nuevo proceso antes de iniciarlo. Proporciona una forma más flexible y poderosa de construir procesos, permitiendo establecer el entorno, redirigir flujos de entrada/salida y configurar otras opciones.

- En Kotlin, podemos crear y gestionar procesos utilizando la clase ProcessBuilder.

Process.kt

```kotlin
import java.io.BufferedReader
import java.io.File
import java.io.InputStreamReader

fun main(){
    val processBuilder = ProcessBuilder("bash","-c","ls -l")
    //Establecer el directorio de trabajo
    processBuilder.directory(File("C:\\Users\\Usuario\\Desktop\\2DAM-AS-2627\\Kotlin_Practicas\\src\\Tema1"))
    // Redirigir errores al flujo de salida
    processBuilder.redirectErrorStream(true)
    try {

        // Iniciamos el proceso, start devuelve un objeto Process
        val process = processBuilder.start()

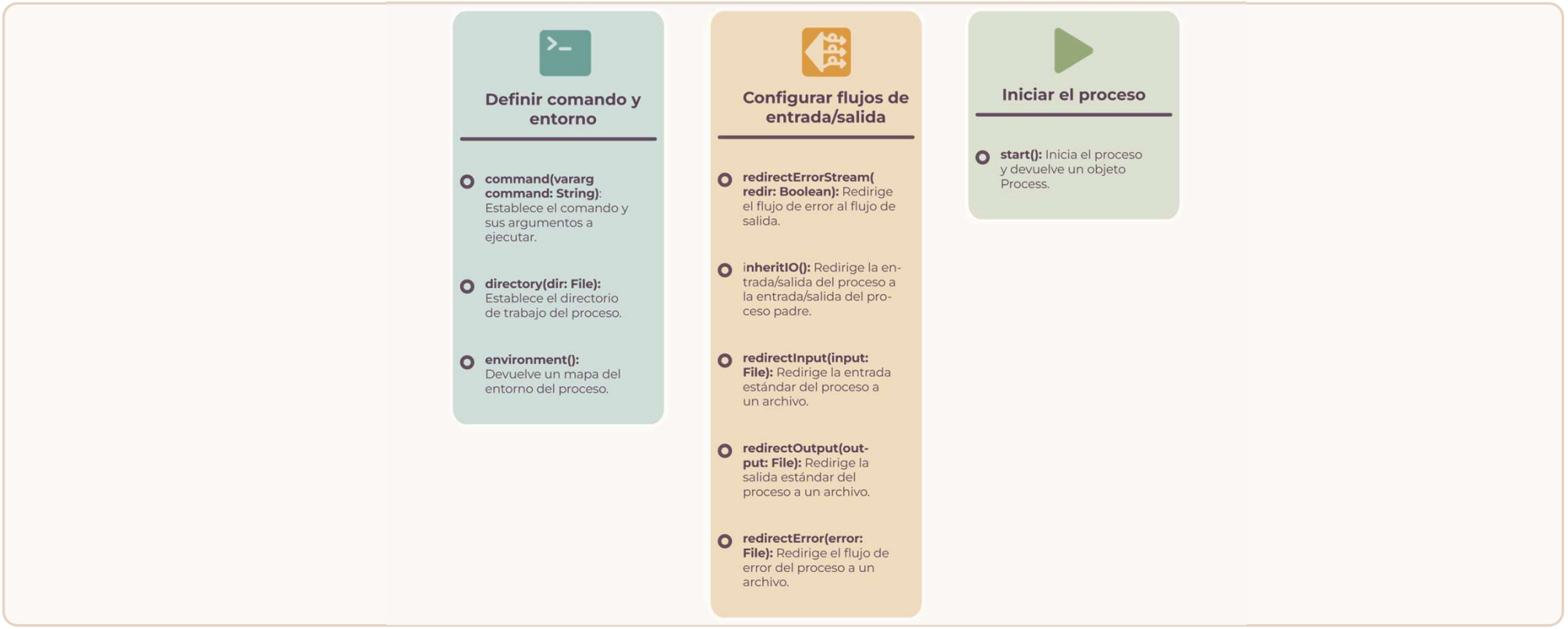
        //Leer la salida del proceso
        val reader = BufferedReader(InputStreamReader(process.inputStream))
        var line:String?
        while(reader.readLine().also {line = it}!= null){
            println(line)
        }
        //Esperar a que el proceso finalice
        val exitCode = process.waitFor()
        println("Proceso terminado: $exitCode")
    }catch(e: Exception){
        e.printStackTrace()
    }
}
```

TEMA · EXPLICACIÓN

Métodos de ProcessBuilder

PROMETEO by thePower

CONCEPTO



TEMA · ACTIVIDAD

Ejercicio 7

PROMETEO by thePower

5 minutos

Enunciado: Escribe un programa que ejecute el comando echo hola y redirija su salida como entrada a otro proceso que ejecute tr a-z A-Z (convierte a mayúsculas).

Comandos: bash echo hola y tr a-z A-Z

Requisitos:

Usar streams para conectar procesos.

TEMA · ACTIVIDAD

PROMETEO by thePower

Ejercicio 8: Monitorización de procesos

10 minutos

Enunciado: Monitorizar procesos más consumidores de CPU (los 3 primeros)

Comandos:

bash ps -eo pid,comm,%cpu --sort=-%cpu | head -n 3

Requisitos:

Lanzar y monitorizar el proceso.
Mostrar el uso de CPU.

TEMA · EXPLICACIÓN

PROMETEO by thePower

CONCEPTO

La clase Runtime

La clase Runtime para la creación de procesos

La principal diferencia entre ProcessBuilder y Runtime radica en el nivel de control que ofrecen sobre el proceso. De hecho, internamente Runtime.exec() crea un ProcessBuilder.

ProcessBuilder permite una configuración más detallada del entorno del proceso, incluyendo la redirección de flujos de entrada/salida y la modificación del entorno del proceso. Por otro lado, Runtime es más simple y directo, pero ofrece menos opciones de configuración.

La decisión de cuándo utilizar una opción u otra depende de los requisitos específicos de la aplicación. Si se necesita un control fino sobre el proceso y su entorno, ProcessBuilder es la mejor opción. Si se busca una solución rápida y sencilla, Runtime puede ser suficiente.

Runtime.kt

```kotlin
import java.io.BufferedReader
import java.io.InputStreamReader

fun main() {
    try {
        val process = Runtime.getRuntime().exec(arrayOf("ls", "-l"))
        //La versión Runtime.exec(String), que recibe el comando completo en una sola cadena,
        // está marcada como obsoleta (deprecated) desde Java 18,
        // porque divide el texto por espacios y falla con rutas o argumentos que los contienen.
        // Hay que usar la versión que recibe un array (exec(arrayOf("ls", "-l"))) o,
        // mejor aún, ProcessBuilder.
        val reader = BufferedReader(InputStreamReader(process.inputStream))
        var line: String?
        while (reader.readLine().also { line = it } != null) {
            println(line)
        }
        val exitCode = process.waitFor()
        println("Proceso terminado con código de salida: $exitCode")
    } catch (e: Exception) {
        e.printStackTrace()
    }
}
```

TEMA · EXPLICACIÓN

La clase ProcessHandle

PROMETEO by thePower

CONCEPTO

La clase ProcessHandle

La clase ProcessHandle proporciona una forma de interactuar con procesos en ejecución. Permite obtener información sobre un proceso, como su ID, estado y tiempo de ejecución. Aquí hay un ejemplo de cómo usar la clase ProcessHandle.

En este ejemplo, creamos un proceso que ejecuta el comando sleep 10. Utilizamos ProcessBuilder para iniciar el proceso y luego obtenemos su ProcessHandle para acceder a información sobre el proceso, como su ID.

ProcessHandle.kt

```
import java.lang.ProcessBuilder

fun main() {
    // Crear un proceso para ejecutar el comando 'sleep 10'
    val processBuilder = ProcessBuilder("bash","-c","sleep", "10")
    try {
        val process = processBuilder.start() // Iniciar el proceso
        val processHandle = process.toHandle() // Obtener el ProcessHandle del proceso

        println("Proceso iniciado con PID: ${processHandle.pid()}")

        // Esperar a que el proceso termine
        val exitCode = process.waitFor()
        println("Proceso terminado con código de salida: $exitCode")
    } catch (e: Exception) {
        e.printStackTrace()
    }
}
```

TEMA · EXPLICACIÓN

PROMETEO by thePower

CONCEPTO

Métodos de ProcessHandle



Obtener información del proceso

- pid(): Devuelve el ID del proceso.
- isAlive(): Devuelve true si el proceso aún está en ejecución.
- info(): Devuelve un objeto ProcessHandle.Info con información sobre el proceso.



Gestionar jerarquía de procesos

- children(): Devuelve un flujo de los procesos hijos del proceso.
- descendants(): Devuelve un flujo con todos los descendientes (hijos, nietos...).
- parent(): Devuelve un Optional<ProcessHandle> con el proceso padre, si existe.



Controlar ciclo de vida

- destroy(): Solicita la finalización del proceso.
- destroyForcibly(): Fuerza la finalización inmediata del proceso.
- onExit(): Devuelve un CompletableFuture<ProcessHandle> que se completa cuando el proceso termina.

TEMA · EXPLICACIÓN

PROMETEO by thePower

CONCEPTO

Lanzamiento de procesos

- Lanzar un proceso en **segundo plano** significa que el programa padre no se queda esperando a que termine: sigue trabajando y, si le interesa, recibe un aviso cuando el hijo acaba. Si no nos interesa su salida, podemos descartarla con ProcessBuilder.Redirect.DISCARD (equivale a redirigir a /dev/null en Unix o a NUL en Windows). (Ver ejemplo Tema1/SegundoPlano.kt)

- Lanzar un proceso en **primer plano** significa que el programa padre espera a que termine antes de continuar. Si además queremos que el proceso hijo use directamente la consola del padre (para mostrar su salida o leer del teclado), podemos usar inheritIO(). (Ver ejemplo Tema1/PrimerPlano.kt)

⚠️ Si un proceso hijo escribe mucho en su salida y nadie la lee (ni se redirige ni se hereda), el búfer del sistema operativo se llena y el hijo se queda bloqueado para siempre. Por eso, siempre hay que leer inputStream/errorStream, **redirigirlos a un fichero**, **descartarlos (Redirect.DISCARD) o heredarlos (inheritIO()).**

TEMA · EXPLICACIÓN

PROMETEO by thePower

CONCEPTO

Ejecución de programas o scripts externos

Para ejecutar programas o scripts externos desde una aplicación Kotlin, podemos utilizar la clase ProcessBuilder.

En este ejemplo, creamos un proceso que ejecuta un script externo ubicado en path/to/your/script.sh. Utilizamos ProcessBuilder para configurar y lanzar el proceso, y luego leemos su salida.

Esto nos puede servir para ejecutar cualquier programa o script externo desde nuestra aplicación Kotlin, lo que nos permite integrar funcionalidades adicionales o automatizar tareas.

EjecucionScripts.kt

```kotlin
fun main() {
    // Crear un proceso para ejecutar un script externo
    val processBuilder = ProcessBuilder("path/to/your/script.sh")
    /* Si queremos pasar algo como parámetro al script:
       val processBuilder = ProcessBuilder("path/to/your/script.sh", "arg1", "arg2") */
    processBuilder.directory(File("/path/to/your/script/directory")) // Establecer el directorio de trabajo
    processBuilder.redirectErrorStream(true) // Redirigir errores al flujo de salida

    try {
        val process = processBuilder.start() // Iniciar el proceso

        // Leer la salida del proceso
        val reader = BufferedReader(InputStreamReader(process.inputStream))
        var line: String?
        while (reader.readLine().also { line = it } != null) {
            println(line)
        }

        // Esperar a que el proceso termine
        val exitCode = process.waitFor()
        println("Proceso terminado con código de salida: $exitCode")
    } catch (e: Exception) {
        e.printStackTrace()
    }
}
```

TEMA · ACTIVIDAD

PROMETEO by thePower

Ejercicio 9: Crear un proceso hijo que ejecute otro programa Kotlin

15 minutos

Enunciado: Escribe dos programas Kotlin: el primero debe lanzar el segundo como proceso hijo y mostrar su salida.

Requisitos:

El programa hijo debe estar compilado previamente como un .jar ejecutable.
El lanzador debe mostrar la salida del hijo y su código de salida.

Resumen

Creación y Gestión de Procesos

- **Clase Process:** representa un proceso del sistema operativo en ejecución y permite interactuar con él (entrada, salida y finalización).
- **Clase ProcessBuilder:** facilita la configuración y creación de nuevos procesos externos de forma flexible.
- **Clase Runtime para creación de procesos:** permite ejecutar programas externos mediante métodos como exec().
- **Clase ProcessHandle:** ofrece información y control avanzado sobre procesos, como PID, estado y procesos hijos.
- **Lanzamiento de procesos en segundo plano:** ejecuta un proceso sin bloquear la ejecución de la aplicación principal.
- **Lanzamiento de procesos en primer plano**: ejecuta un proceso esperando su finalización antes de continuar.
- **Ejecución de programas o scripts externos**: consiste en invocar aplicaciones o scripts del sistema desde un programa Java.

Sección 3.2

Comunicación entre procesos

03.2

TEMA · EXPLICACIÓN

PROMETEO by thePower

CONCEPTO

Comunicación entre procesos

Existen diversas formas de comunicación entre procesos, entre las que se incluyen:

- **Pipes:** Permiten la comunicación unidireccional entre procesos. Un proceso puede escribir en un pipe y otro proceso puede leer de él.
- **Sockets:** Proporcionan una comunicación bidireccional entre procesos, incluso si están en diferentes máquinas.
- **Memoria compartida:** Permite que varios procesos accedan a la misma región de memoria, lo que facilita la comunicación rápida entre ellos.

TEMA · EXPLICACIÓN

Pipes

PROMETEO by thePower

CONCEPTO

- Los pipes son una forma de comunicación unidireccional entre procesos. Permiten que un proceso escriba datos en un pipe, que luego pueden ser leídos por otro proceso. En Kotlin, se pueden utilizar pipes a través de flujos de entrada/salida.

```kotlin
import java.io.BufferedReader
import java.io.InputStreamReader
import java.io.OutputStreamWriter
import java.lang.ProcessBuilder

fun main() {
    // Crear un proceso para ejecutar el comando 'grep "kotlin"'
    val processBuilder = ProcessBuilder("bash", "-c","grep kotlin")
    val process = processBuilder.start()

    // Escribir en el pipe de entrada del proceso
    val writer = OutputStreamWriter(process.outputStream)
    writer.write("kotlin es un lenguaje de programación\n")
    writer.write("java es otro lenguaje de programación\n")
    writer.flush()
    writer.close()

    // Leer la salida del proceso
    val reader = BufferedReader(InputStreamReader(process.inputStream))
    var line: String?
    while (reader.readLine().also { line = it } != null) {
        println(line)
    }

    // Esperar a que el proceso termine
    val exitCode = process.waitFor()
    println("Proceso terminado con código de salida: $exitCode")
}
```

TEMA · EXPLICACIÓN

Sockets

PROMETEO by thePower

CONCEPTO

- A diferencia de las pipes, los Sockets proporcionan una comunicación bidireccional entre procesos, incluso si están en diferentes máquinas. Los sockets permiten que un proceso envíe y reciba datos a través de una conexión de red.

- Para probar el uso de sockets, vamos a ir a los ejemplos de código Sockets.kt y Sockets02.kt.

TEMA · EXPLICACIÓN

PROMETEO by thePower

CONCEPTO

Memoria compartida

- Otra opción de comunicación entre procesos es la memoria compartida. Al utilizar memoria compartida, varios procesos pueden acceder a la misma región de memoria, lo que facilita la comunicación rápida entre ellos, ya que los datos no tienen que copiarse de un proceso a otro.

- En sistemas operativos como Linux, la memoria compartida se gestiona con llamadas al sistema como shmget/shmat o shm_open/mmap. La JVM no permite usar esas llamadas directamente, pero sí ofrece ficheros mapeados en memoria (MappedByteBuffer): el sistema operativo carga el fichero en memoria y todos los procesos que lo mapean comparten las mismas páginas. Es la forma habitual de usar memoria compartida desde Java y Kotlin.

- Ejemplos: EscritorMemoriaCompartida.kt y LectorMemoriaCompartida.kt

⚠️ **La memoria compartida necesita sincronización**
Si varios procesos leen y escriben a la vez en la misma zona de memoria, pueden producirse condiciones de carrera. En un caso real, el acceso debería protegerse, por ejemplo, con un bloqueo de fichero (FileChannel.lock()), que veremos en el siguiente apartado.

Sección 3.3

Sincronización entre procesos

03.3

TEMA · EXPLICACIÓN

PROMETEO by thePower

CONCEPTO

Sincronización entre procesos

- La sincronización de procesos se refiere a la coordinación de la ejecución de múltiples procesos para garantizar que accedan a recursos compartidos de manera controlada y evitar condiciones de carrera. Existen varias técnicas para lograr la sincronización, incluyendo semáforos, mutexes y monitores.

Procesos e hilos en los ejemplos

- Los semáforos, mutexes y monitores son conceptos generales que el sistema operativo ofrece tanto para procesos como para hilos. Sin embargo, las clases de Java/Kotlin que los implementan (Semaphore, ReentrantLock...) solo funcionan dentro de una misma JVM, es decir, entre hilos. Por eso, en los siguientes ejemplos usaremos hilos para simular procesos concurrentes. Los hilos se estudian en profundidad en el Tema 2.

- Para sincronizar procesos distintos desde la JVM, la herramienta más habitual es el bloqueo de ficheros (FileChannel.lock()), que veremos un poco más adelante.

TEMA · EXPLICACIÓN

Semáforos

PROMETEO by thePower

CONCEPTO

Un semáforo es un contador que controla cuántos procesos o hilos pueden acceder a la vez a un recurso compartido. Tiene dos operaciones básicas:

- acquire() (también llamada wait o P): si el contador es mayor que 0, lo decrementa y continúa; si es 0, se bloquea hasta que otro libere un permiso.
- release() (también llamada signal o V): incrementa el contador y despierta a alguno de los que estaban esperando.

Un semáforo inicializado a 1 (semáforo binario) solo deja pasar a uno a la vez; uno inicializado a N permite N accesos simultáneos.

Ejemplo en Semaforo.kt

TEMA · EXPLICACIÓN

Mutexes

PROMETEO by thePower

CONCEPTO

- Los mutexes (mutual exclusion) son similares a un semáforo binario, pero con una diferencia importante: el mutex tiene propietario. Solo quien lo ha bloqueado puede desbloquearlo.
- Un mutex puede estar bloqueado o desbloqueado; cuando alguien lo bloquea, los demás deben esperar hasta que sea desbloqueado.

⚠️ **Locks y corrutinas**
Los locks de Java (ReentrantLock) pertenecen al hilo que los adquiere. Una corrutina puede suspenderse y reanudarse en otro hilo, por lo que no deben combinarse con funciones de suspensión como delay() dentro de un withLock. Para corrutinas existe kotlinx.coroutines.sync.Mutex, que veremos en el Tema 2.

- Ejemplo en Mutexes.kt

TEMA · EXPLICACIÓN

PROMETEO by thePower

CONCEPTO

Monitores

- Los monitores son una abstracción de alto nivel para la sincronización. Un monitor es un objeto que encapsula un recurso compartido y proporciona métodos para acceder a él de manera segura.
- Los monitores utilizan un mutex y variables de condición para garantizar que solo uno pueda acceder al recurso a la vez y para que los demás esperen de forma ordenada.

TEMA · EXPLICACIÓN

PROMETEO by thePower

CONCEPTO

Bloqueo de ficheros

Sincronización entre procesos con bloqueo de ficheros

- Cuando los que compiten por un recurso son procesos distintos (por ejemplo, dos programas que escriben en el mismo fichero), podemos usar el bloqueo de ficheros que ofrece el sistema operativo a través de FileChannel.lock()

- Ejemplo BloqueoDeFichertos.kt

PROMETEO
by thePower

SECCIÓN 4

Depuración y Documentación de Aplicaciones Multiproceso

04

TEMA · EXPLICACIÓN

PROMETEO by thePower

CONCEPTO

Técnicas básicas de depuración para apps multiproceso

Depurar aplicaciones multiproceso puede ser un desafío debido a la concurrencia y la interacción entre los procesos. Algunas técnicas básicas incluyen:

- Uso de herramientas de depuración específicas: El depurador del IDE (IntelliJ IDEA) permite establecer puntos de interrupción y seguir la ejecución paso a paso, e incluso conectarse a un proceso ya en marcha (Attach to Process). Para programas nativos se usan herramientas como GDB (GNU Debugger).
- Registros detallados (logging): Implementar un sistema de log robusto que registre eventos importantes, errores y estados de los procesos. Esto ayuda a rastrear el comportamiento de la aplicación en tiempo real y a identificar problemas.
- Pruebas unitarias y de integración: Crear pruebas específicas para cada proceso y para la interacción entre ellos. Esto ayuda a asegurar que cada componente funcione correctamente de manera aislada y en conjunto.
- Análisis de concurrencia: Utilizar herramientas que detecten condiciones de carrera, bloqueos y otros problemas relacionados con la concurrencia.
- Simulación de carga: Probar la aplicación bajo diferentes condiciones de carga para identificar problemas de rendimiento y escalabilidad.
- Revisión de código: Realizar revisiones de código entre pares para identificar posibles errores y mejorar la calidad del código.
- Monitoreo en tiempo real: Implementar sistemas de monitoreo que permitan observar el comportamiento de los procesos en tiempo real, facilitando la identificación de problemas.
- Uso de contenedores: Ejecutar procesos en contenedores (como Docker) para aislar el entorno y facilitar la depuración.

TEMA · EXPLICACIÓN

PROMETEO by thePower

CONCEPTO

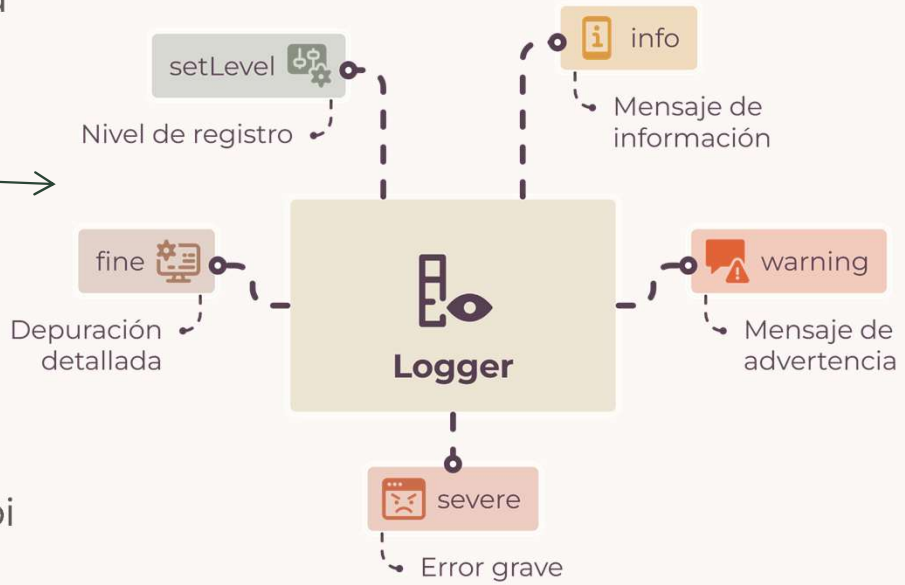
La clase Logger

La clase Logger es parte del paquete java.util.logging, que proporciona una forma sencilla de agregar capacidades de logging a las aplicaciones Java y Kotlin.

Sus métodos principales incluyen:

Podemos configurar diferentes niveles de logging para controlar la cantidad de información registrada, lo que es útil para depurar sin sobrecargar los registros con demasiados detalles.

Documentación oficial de java.util.logging: https://docs.oracle.com/en/java/javase/21/docs/api/java.logging/java/util/logging/package-summary.html



TEMA · EXPLICACIÓN

PROMETEO by thePower

CONCEPTO

Ejemplo de logging en Kotlin

Vamos a ver un ejemplo sencillo de cómo implementar un sistema de logging en una aplicación multiproceso utilizando Kotlin y la biblioteca java.util.logging.

- En este ejemplo, se crea un logger que escribe en un archivo llamado app.log. Cada proceso simulado registra su inicio y finalización, lo que facilita el seguimiento de la ejecución de los procesos.
- La clase FileHandler se utiliza para escribir los registros en un archivo, y SimpleFormatter formatea los mensajes de registro de manera legible. Cada hilo (proceso simulado) registra su actividad, lo que ayuda a depurar y entender el flujo de la aplicación multiproceso.

```
import java.util.logging.FileHandler
import java.util.logging.Logger
import java.util.logging.SimpleFormatter
import kotlin.concurrent.thread

fun main() {
    val logger = Logger.getLogger("MyLogger")
    val fileHandler = FileHandler("app.log", true)
    fileHandler.formatter = SimpleFormatter()
    logger.addHandler(fileHandler)

    // Simulación de procesos concurrentes
    repeat(5) { i ->
        thread {
            logger.info("Iniciando proceso $i")
            Thread.sleep((1000..3000).random().toLong())
            logger.info("Finalizando proceso $i")
        }
    }
}
```

TEMA · ACTIVIDAD

PROMETEO by thePower

Ejercicio 10: Logging y depuración en procesos concurrentes

15 minutos

Enunciado: Escribe un programa que lance varios hilos, cada uno de los cuales escribe mensajes en un log usando java.util.logging.

Nota: El uso de hilos (thread) para simular procesos concurrentes se estudiará en profundidad en el siguiente tema. Este ejercicio se incluye como ejemplo avanzado para quien quiera experimentar.

Requisitos:

Usar logging y simular procesos concurrentes.

TEMA · EXPLICACIÓN

PROMETEO by thePower

CONCEPTO

Pruebas unitarias en Kotlin

- Las pruebas unitarias son una parte esencial del desarrollo de software, ya que permiten verificar que cada componente de la aplicación funcione correctamente de forma aislada. En Kotlin, podemos utilizar bibliotecas como JUnit y MockK para facilitar la creación y ejecución de pruebas unitarias. Nosotros vamos a usar JUnit.

Para eso hay que añadir la dependencia de Junit:

- Si usas Maven en el pom.xml

```
<dependencies>
  <dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter</artifactId>
    <version>5.13.4</version>
    <scope>test</scope>
  </dependency>
</dependencies>
```

- Si usas Gradle:

```
dependencies {
  testImplementation("org.junit.jupiter:junit-jupiter:5.13.4")
}

tasks.test {
  useJUnitPlatform()
}
```

- Y si no usas ninguno de los 2, tal y como debería ser el caso, mira la siguiente diapositiva.

TEMA · EXPLICACIÓN

Trabajando sin Maven ni Gradle

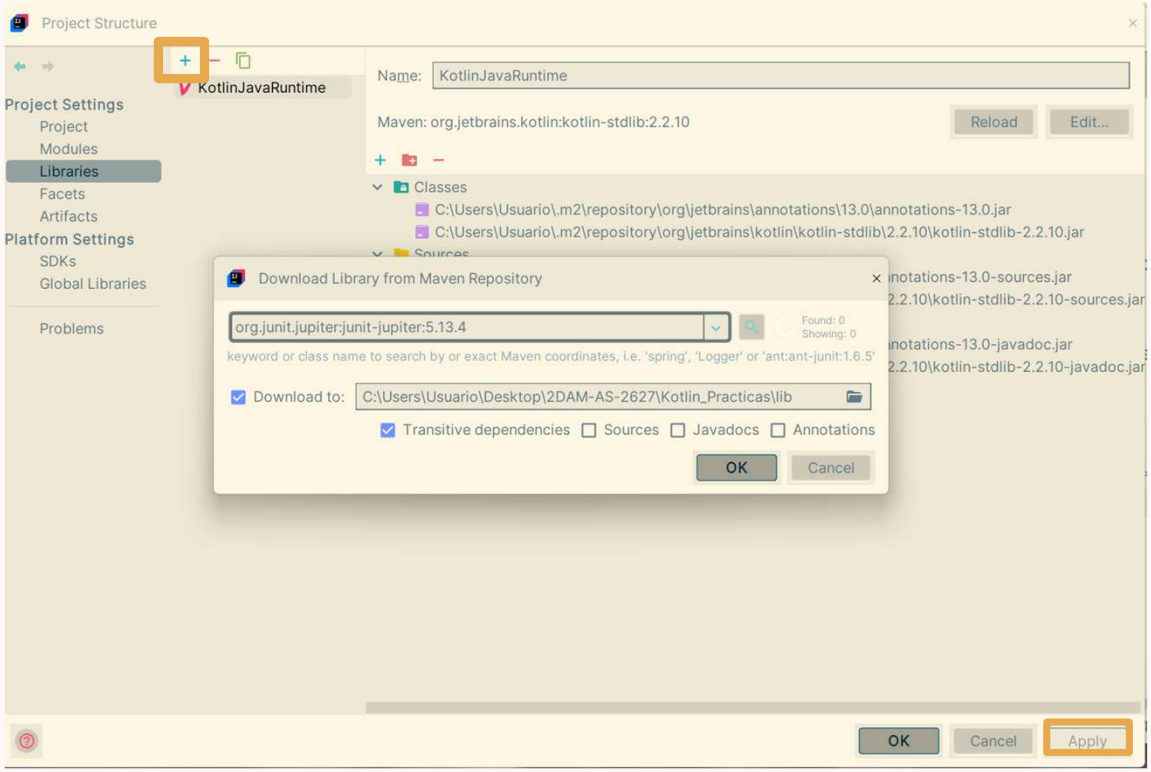
PROMETEO by thePower

CONCEPTO

File → Project Structure → Libraries→+

org.junit.jupiter:junit-jupiter:5.13.4

Ejemplo en: PruebaUnitaria.kt



PROMETEO by thePower

CONCEPTO

Pruebas unitarias en aplicaciones multiproceso

En aplicaciones multiproceso, las pruebas unitarias pueden ser más complejas debido a la concurrencia y la interacción entre procesos. Algunas estrategias para manejar esto incluyen:

- Aislamiento de procesos: Probar cada proceso de forma independiente para asegurarse de que funcione correctamente.
- Simulación de interacciones: Utilizar mocks para simular la comunicación entre procesos.
- Pruebas de integración: Complementar las pruebas unitarias con pruebas de integración que verifiquen la interacción entre múltiples procesos.
- Uso de entornos controlados: Ejecutar las pruebas en entornos controlados para minimizar la interferencia externa.
- Monitoreo de recursos: Asegurarse de que los procesos no consuman más recursos de los esperados durante las pruebas.

TEMA · EXPLICACIÓN

PROMETEO by thePower

CONCEPTO

Pruebas de Integración

- Las pruebas de integración son esenciales para verificar que los diferentes componentes de una aplicación multiproceso funcionen correctamente juntos. En Kotlin, podemos utilizar JUnit junto con bibliotecas como Testcontainers para facilitar la creación y ejecución de pruebas de integración.

- Para realizar pruebas de integración en aplicaciones multiproceso, es importante considerar aspectos como la comunicación entre procesos, la sincronización y el manejo de recursos compartidos. Ver ejemplo: PruebasIntegracion.kt

- A diferencia de las pruebas unitarias, que se centran en componentes individuales, las pruebas de integración aseguran que los procesos interactúen correctamente entre sí.

- Es importante ejecutar las pruebas de integración en un entorno que refleje lo más fielmente posible el entorno de producción, para identificar problemas que puedan surgir debido a la interacción entre procesos.

TEMA · EXPLICACIÓN

PROMETEO by thePower

CONCEPTO

Depuración en tiempo real

Depurar aplicaciones multiproceso en tiempo real puede ser complicado debido a la naturaleza concurrente de los procesos. Algunas estrategias para facilitar la depuración en tiempo real incluyen:

- Monitoreo de procesos: Utilizar herramientas de monitoreo para observar el estado de los procesos en tiempo real.
- Logging en tiempo real: Implementar un sistema de logging que permita ver los registros a medida que se generan.
- Depuración remota: Configurar la aplicación para permitir la depuración remota, lo que facilita la inspección de procesos en ejecución.
- Análisis de rendimiento: Utilizar herramientas de análisis de rendimiento para identificar cuellos de botella y problemas de rendimiento en tiempo real.
- Visualización de datos: Implementar dashboards que muestren métricas clave y el estado de los procesos en tiempo real.

TEMA · EXPLICACIÓN

PROMETEO by thePower

CONCEPTO

Herramientas de depuración

Algunas herramientas populares para la depuración de aplicaciones multiproceso incluyen:

- Depurador de IntelliJ IDEA: Permite depurar programas Kotlin/Java, incluidos procesos remotos o ya en ejecución (Attach to Process).
- Herramientas del JDK: jps (lista los procesos Java en ejecución), jcmd y jstack (vuelcan el estado de los hilos de una JVM) y JDK Mission Control / Java Flight Recorder (perfilado).
- GDB: Un depurador potente para programas escritos en C/C++.
- Valgrind: Herramienta para detectar fugas de memoria y errores de memoria.
- strace: Utilidad para rastrear llamadas al sistema y señales.
- ltrace: Similar a strace, pero rastrea llamadas a funciones de bibliotecas.
- htop: Monitor interactivo de procesos en tiempo real.
- Wireshark: Herramienta para analizar el tráfico de red, útil en aplicaciones distribuidas.
- Prometheus y Grafana: Herramientas para monitoreo y visualización de métricas en tiempo real.

TEMA · EXPLICACIÓN

PROMETEO by thePower

CONCEPTO

Buenas prácticas de depuración

- Reproducibilidad: Asegurarse de que los problemas puedan ser reproducidos de manera consistente para facilitar su análisis.
- Aislamiento de problemas: Intentar aislar el problema a un proceso o componente específico.
- Documentación de hallazgos: Mantener un registro de los problemas encontrados y las soluciones implementadas.
- Comunicación efectiva: Fomentar la comunicación entre los miembros del equipo para compartir conocimientos y soluciones.
- Iteración: La depuración es un proceso iterativo; es importante revisar y ajustar las estrategias según sea necesario.

TEMA · EXPLICACIÓN

PROMETEO by thePower

CONCEPTO

Documentación de aplicaciones multiproceso

Documentar aplicaciones multiproceso es crucial para mantener la claridad y facilitar el mantenimiento. Para una documentación efectiva, se deben considerar los siguientes aspectos:

- Arquitectura del sistema: Describir la arquitectura general de la aplicación, incluyendo cómo los procesos interactúan entre sí y con otros sistemas.
- Diagrama de flujo: Incluir diagramas que representen el flujo de datos y la comunicación entre procesos.
- Descripción de procesos: Documentar cada proceso individualmente, incluyendo su propósito, entradas, salidas y dependencias.
- Manejo de errores: Explicar cómo se manejan los errores y las excepciones en cada proceso.
- Configuración: Proporcionar detalles sobre la configuración necesaria para ejecutar la aplicación, incluyendo variables de entorno y parámetros de inicio.
- Guías de uso: Incluir instrucciones claras sobre cómo ejecutar, detener y monitorear la aplicación.
- Ejemplos de código: Proporcionar ejemplos de código para ilustrar cómo interactuar con los procesos.
- Historial de cambios: Mantener un registro de cambios para documentar las modificaciones realizadas en la aplicación a lo largo del tiempo.
- Referencias externas: Incluir enlaces a documentación externa relevante, como bibliotecas o frameworks utilizados en la aplicación.
- Comentarios en el código: Asegurarse de que el código esté bien comentado para facilitar la comprensión de su funcionamiento.

¿Un poco se parece a lo que hace vuestra profe con vosotros no?

TEMA · ACTIVIDAD

PROMETEO by thePower

Ejercicio 11: Documentar el flujo de un programa multiproceso con validación

15 minutos

Enunciado: Escribe un programa que cree un archivo temporal, escriba varias líneas en él usando un proceso, luego use otro proceso para contar las líneas (wc -l), y finalmente lea y muestre el contenido. Documenta detalladamente el flujo del programa usando comentarios y un diagrama de flujo.

Comandos:

bash: printf (escribir), wc -l (contar) y cat (leer)

Requisitos:

- *Documentar el código y el flujo.*
- *Incluir un diagrama de flujo.*

